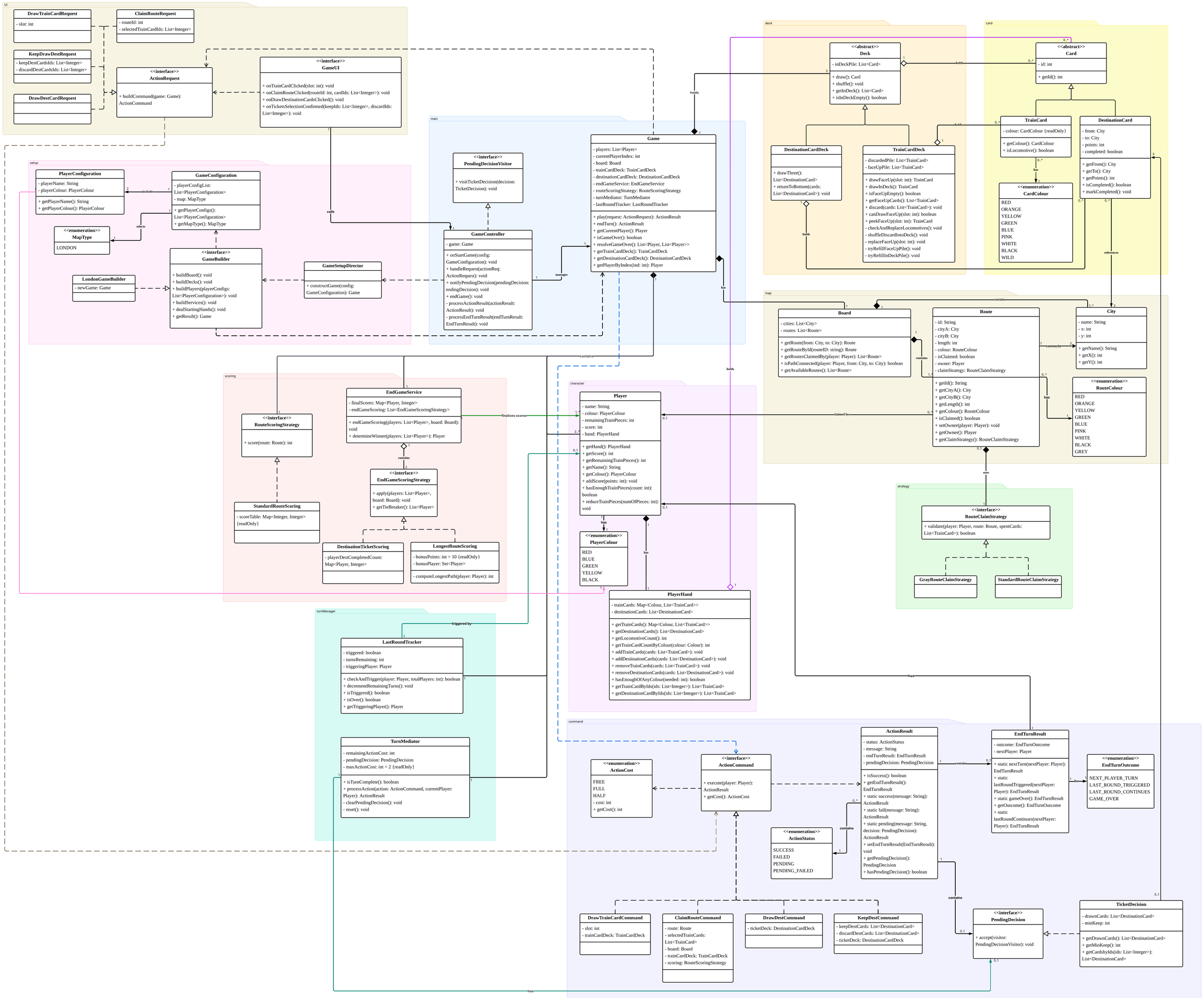

UI
DrawTrainCardRequest
- slot: int
ClaimRouteRequest
- routeId: int
- selectedTrainCardIds: List<Integer>
KeepDrawDestRequest
- keepDestCardsIds: List<Integer>
- discardDestCardsIds: List<Integer>
DrawDestCardRequest
<<interface>>
ActionRequest
+ buildCommand(game: Game): ActionCommand
<<interface>>
GameUI
+ onTrainCardClicked(slot: int): void
+ onClaimRouteClicked(routeId: int, cardIds: List<Integer>): void
+ onDrawDestinationCardsClicked(): void
+ onTicketsSelectionConfirmed(keepIds: List<Integer>, discardIds: List<Integer>): void
calls
holds
setup
PlayerConfiguration
- playerName: String
- playerColour: PlayerColour
+ getPlayerName(): String
+ getPlayerColour(): PlayerColour
includes
GameConfiguration
- playerConfigList: List<PlayerConfiguration>
- map: MapType
+ getPlayerConfigs(): List<PlayerConfiguration>
+ getMapType(): MapType
selects
<<enumeration>>
MapType
LONDON
LondonGameBuilder
- newGame: Game
<<interface>>
GameBuilder
+ buildBoard(): void
+ buildDecks(): void
+ buildPlayers(playerConfigs: List<PlayerConfiguration>): void
+ buildServices(): void
+ dealStartingHands(): void
+ getResult(): Game
GameSetupDirector
+ constructGame(config: GameConfiguration): Game
main
<<interface>>
PendingDecisionVisitor
+ visitTicketDecision(decision: TicketDecision): void
GameController
- game: Game
+ onStartGame(config: GameConfiguration): void
+ handleRequest(actionReq: ActionRequest): void
+ notifyPendingDecision(pendingDecision: endingDecision): void
+ endGame(): void
- processActionResult(actionResult: ActionResult): void
- processEndTurnResult(endTurnResult: EndTurnResult): void
manages
Game
- players: List<Player>
- currentPlayerIndex: int
- board: Board
- trainCardDeck: TrainCardDeck
- destinationCardDeck: DestinationCardDeck
- endGameService: EndGameService
- routeScoringStrategy: RouteScoringStrategy
- turnMediator: TurnMediator
- lastRoundTracker: LastRoundTracker
+ play(request: ActionRequest): ActionResult
+ endTurn(): ActionResult
+ getCurrentPlayer(): Player
+ isGameOver(): boolean
+ resolveGameOver(): List<Player, List<Player>>
+ getTrainCardDeck(): TrainCardDeck
+ getDestinationCardDeck(): DestinationCardDeck
+ getPlayerByIndex(ind: int): Player
has
deck
<<abstract>>
Deck
- inDeckPile: List<Card>
+ draw(): Card
+ shuffle(): void
+ getInDeck(): List<Card>
+ isInDeckEmpty(): boolean
holds
DestinationCardDeck
+ drawThree(): List<DestinationCard>
+ returnToBottom(cards: List<DestinationCard>): void
TrainCardDeck
- discardedPile: List<TrainCard>
- faceUpPile: List<TrainCard>
+ drawFaceUp(slot: int): TrainCard
+ drawInDeck(): TrainCard
+ isFaceUpEmpty(): boolean
+ getFaceUpCards(): List<TrainCard>
+ discard(cards: List<TrainCard>): void
+ canDrawFaceUp(slot: int): boolean
+ peekFaceUp(slot: int): TrainCard
- checkAndReplaceLocomotives(): void
- shuffleDiscardIntoDeck(): void
- replaceFaceUp(slot: int): void
- tryRefillFaceUpPile(): void
- tryRefillInDeckPile(): void
card
<<abstract>>
Card
- id: int
+ getId(): int
TrainCard
- colour: CardColour {readOnly}
+ getColour(): CardColour
+ isLocomotive(): boolean
DestinationCard
- from: City
- to: City
- points: int
- completed: boolean
+ getFrom(): City
+ getTo(): City
+ getPoints(): int
+ isCompleted(): boolean
+ markCompleted(): void
has
<<enumeration>>
CardColour
RED
ORANGE
YELLOW
GREEN
BLUE
PINK
WHITE
BLACK
WILD
references
map
Board
- cities: List<City>
- routes: List<Route>
+ getRoute(from: City, to: City): Route
+ getRouteById(routeID: string): Route
+ getRoutesClaimedBy(player: Player): List<Route>
+ isPathConnected(player: Player, from: City, to: City): boolean
+ getAvailableRoutes(): List<Route>
contains
Route
- id: String
- cityA: City
- cityB: City
- length: int
- colour: RouteColour
- isClaimed: boolean
- owner: Player
- claimStrategy: RouteClaimStrategy
+ getId(): String
+ getCityA(): City
+ getCityB(): City
+ getLength(): int
+ getColour(): RouteColour
+ isClaimed(): boolean
+ setOwner(player: Player): void
+ getOwner(): Player
+ getClaimStrategy(): RouteClaimStrategy
connects
City
- name: String
- x: int
- y: int
+ getName(): String
+ getX(): int
+ getY(): int
<<enumeration>>
RouteColour
RED
ORANGE
YELLOW
GREEN
BLUE
PINK
WHITE
BLACK
GREY
claimed by
uses
strategy
<<interface>>
RouteClaimStrategy
+ validate(player: Player, route: Route, spentCards: List<TrainCard>): boolean
GrayRouteClaimStrategy
StandardRouteClaimStrategy
scoring
<<interface>>
RouteScoringStrategy
+ score(route: Route): int
StandardRouteScoring
- scoreTable: Map<Integer, Integer> {readOnly}
EndGameService
- finalScores: Map<Player, Integer>
- endGameScoring: List<EndGameScoringStrategy>
+ endGameScoring(players: List<Player>, board: Board): void
+ determineWinner(players: List<Player>): Player
finalizes scores
<<interface>>
EndGameScoringStrategy
+ apply(players: List<Player>, board: Board): void
+ getTieBreaker(): List<Player>
DestinationTicketScoring
- playerDestCompletedCount: Map<Player, Integer>
LongestRouteScoring
- bonusPoints: int = 10 {readOnly}
- bonusPlayer: Set<Player>
- computeLongestPath(player: Player): int
character
Player
- name: String
- colour: PlayerColour
- remainingTrainPieces: int
- score: int
- hand: PlayerHand
+ getHand(): PlayerHand
+ getScore(): int
+ getRemainingTrainPieces(): int
+ getName(): String
+ getColour(): PlayerColour
+ addScore(points: int): void
+ hasEnoughTrainPieces(count: int): boolean
+ reduceTrainPieces(numOfPieces: int): void
has
<<enumeration>>
PlayerColour
RED
BLUE
GREEN
YELLOW
BLACK
PlayerHand
- trainCards: Map<Colour, List<TrainCard>>
- destinationCards: List<DestinationCard>
+ getTrainCards(): Map<Colour, List<TrainCard>>
+ getDestinationCards(): List<DestinationCard>
+ getLocomotiveCount(): int
+ getTrainCardCountByColour(colour: Colour): int
+ addTrainCards(cards: List<TrainCard>): void
+ addDestinationCards(cards: List<DestinationCard>): void
+ removeTrainCards(cards: List<TrainCard>): void
+ removeDestinationCards(cards: List<DestinationCard>): void
+ hasEnoughOfAnyColour(needed: int): boolean
+ getTrainCardByIds(ids: List<Integer>): List<TrainCard>
+ getDestinationCardByIds(ids: List<Integer>): List<TrainCard>
turnManager
triggered by
LastRoundTracker
- triggered: boolean
- turnsRemaining: int
- triggeringPlayer: Player
+ checkAndTrigger(player: Player, totalPlayers: int): boolean
+ decrementRemainingTurns(): void
+ isTriggered(): boolean
+ isOver(): boolean
+ getTriggeringPlayer(): Player
TurnMediator
- remainingActionCost: int
- pendingDecision: PendingDecision
- maxActionCost: int = 2 {readOnly}
+ isTurnComplete(): boolean
+ processAction(action: ActionCommand, currentPlayer: Player): ActionResult
- clearPendingDecision(): void
- reset(): void
command
<<enumeration>>
ActionCost
FREE
FULL
HALF
- cost: int
+ getCost(): int
<<interface>>
ActionCommand
+ execute(player: Player): ActionResult
+ getCost(): ActionCost
ActionResult
- status: ActionStatus
- message: String
- endTurnResult: EndTurnResult
- pendingDecision: PendingDecision
+ isSuccess(): boolean
+ getEndTurnResult(): EndTurnResult
+ static success(message: String): ActionResult
+ static fail(message: String): ActionResult
+ static pending(message: String, decision: PendingDecision): ActionResult
+ setEndTurnResult(EndTurnResult): void
+ getPendingDecision(): PendingDecision
+ hasPendingDecision(): boolean
contains
<<enumeration>>
ActionStatus
SUCCESS
FAILED
PENDING
PENDING_FAILED
EndTurnResult
- outcome: EndTurnOutcome
- nextPlayer: Player
+ static nextTurn(nextPlayer: Player): EndTurnResult
+ static lastRoundTriggered(nextPlayer: Player): EndTurnResult
+ static gameOver(): EndTurnResult
+ getOutcome(): EndTurnOutcome
+ static lastRoundContinues(nextPlayer: Player): EndTurnResult
<<enumeration>>
EndTurnOutcome
NEXT_PLAYER_TURN
LAST_ROUND_TRIGGERED
LAST_ROUND_CONTINUES
GAME_OVER
DrawTrainCardCommand
- slot: int
- trainCardDeck: TrainCardDeck
ClaimRouteCommand
- route: Route
- selectedTrainCards: List<TrainCard>
- board: Board
- trainCardDeck: TrainCardDeck
- scoring: RouteScoringStrategy
DrawDestCommand
- ticketDeck: DestinationCardDeck
KeepDestCommand
- keepDestCards: List<DestinationCard>
- discardDestCards: List<DestinationCard>
- ticketDeck: DestinationCardDeck
<<interface>>
PendingDecision
+ accept(visitor: PendingDecisionVisitor): void
TicketDecision
- drawnCards: List<DestinationCard>
- minKeep: int
+ getDrawnCards(): List<DestinationCard>
+ getMinKeep(): int
+ getCardsbyIds(ids: List<Integer>): List<DestinationCard>
has